

First Name	Joshua	Family Name	Anto	Student ID No	17988440
Paper Name	Embedded systems design	Paper Code:	ENEL712	Assignment Due Date	6/05/2023
Lecturer:	Jack Li	Tutorial Day	Thursday	Date Submitted	5/06/2023
Tutor:	Biswash	Tutorial Time	2.00pm – 3.00pm	No.Words/Pages	11,242 / 43

In order to ensure fair and honest assessment results for all students, it is a requirement that the work that you hand in for assessment is your own work. If you are uncertain about any of these matters then please discuss them with your lecturer.

Plagiarism and Dishonesty are methods of cheating for the purposes of General Academic Regulations (GAR)

<http://www.aut.ac.nz/calendar>

Assignments will not be accepted if this section is not completed and signed.

Please read the following and **tick** ☒ to indicate your understanding:

- I understand it is my responsibility to keep a copy of my assignment. ☒ **Yes** ☐ **No**
- I have signed and read the **Student's Statement below**. ☒ **Yes** ☐ **No**
- I understand that a software programme (Turnitin) that detects plagiarism and copying may be used on my assignment. ☒ **Yes** ☐ **No**

Student's Statement:

This assessment is entirely my own work and has not been submitted in any other course of study. I have submitted a copy of this assessment to Turnitin, if required.

In this assessment I have acknowledged, to the best of my ability:

- The source of direct quotes from the work of others.
- The ideas of others (includes work from private or professional services, past assessments, other students, books, journals, cut/paste from internet sites and/or other materials)
- The source of diagrams or visual images.

Student's Signature: **Joshua Anto**

Date: **30/05/2023**

The information on this form is collected for the primary purpose of submitting your assignment for assessment. Other purposes of collection include receiving your acknowledgement of plagiarism policies and attending to administrative matters. If you choose not to complete all questions on this form, it may not be possible for the Faculty of Design and Creative Technologies to accept your assignment.

Auckland University of Technology



School of Engineering, Computer and Mathematical Science

ENEL712 Embedded Systems Design

Project Report

Submitted by: Joshua Anto
Student ID: 17988440

6 June 2023

Table of Contents

List of Figures.....	3
List of Tables.....	4
1 Introduction.....	4
1.1 Background.....	4
1.2 Introduction to Microcontroller	5
1.3 AT90USB1287	6
1.3.1 Functions of Microcontrollers.....	7
1.3.2 Evolution of Microcontrollers	8
2 Objectives.....	9
3 Methodologies.....	10
3.1 Setup Tab.....	10
3.1.1 GUI Design	11
3.1.2 COM Port Selection	11
3.1.3 Baud rate selection	11
3.1.4 Serial port connection	11
3.1.5 Database Connection	11
3.2 Digital I/O Tab	12
3.2.1 GUI Design	12
3.2.2 PORTA pins	12
3.2.3 PORTC pins	12
3.3 POTs Tab.....	13
3.3.1 GUI Design	13
3.3.2 Potentiometer functionality	13
3.3.3 Light functionality.....	13
3.3.4 Timer Funtionality.....	14
3.4 Temp Control Tab.....	14
3.4.1 GUI Design.....	14
3.4.2 Temperature Control.....	14
3.4.3 Data Logging.....	14
3.5 Database Server Configuration	15
3.5.1 Database Setup	15
3.5.2 Adding User Account	15
3.5.3 Configure Database	15

3.6	MCU Code	16
3.6.1	Defines and Initializing.....	16
3.6.2	Setup	17
3.6.3	ISR	19
3.6.4	Main Code	20
3.6.5	Instruction Sent code	21
3.7	Equations	22
	<i>PWM Frequency = Clock Frequency(Prescaler * ICR1 + 1)</i>	22
3.8	Design of Flowchart.....	24
3.8.1	Flowchart Components.....	24
	MCU.....	24
	GUI.....	25
	Database.....	26
4	Results	26
4.1	Simulation Results.....	26
4.2	Experimental Results	27
4.3	Comparison of Results	28
5	Discussions	30
6	Conclusion.....	31
7	References	32
8	Appendix.....	32

List of Figures

Figure 1: An Embedded system.....	6
Figure 2: Microchip AT90USB1287	6
Figure 3: ATMEL AT90USB1287 Block diagram	7
Figure 4: AT90USB1287 Lines	7
Figure 5: Setup Tab GUI Design.....	12
Figure 6: Digital I/O Tab GUI Design	13
Figure 7: Temp Control Tab GUI Design.....	15
Figure 8: Define Section of MCU Code	16
Figure 9: Setup Section Code	17
Figure 10: Interrupt Service Routine	19
Figure 11: Light Sensor.....	28
Figure 12: Light simulation.....	29
Figure 13: Motor Simulation.....	29
Figure 14: Pot Simulation.....	30

List of Tables

Table 1: AT90USB1287 Specifications.....	7
Table 2: Main Code.....	20
Table 3: Instruction Sent Code.....	21
Table 4: MCU Code	33
Table 5: GUI Form.cs code.....	36
Table 6: AppBoard Code.....	39

List of Equations

Equation 1: PWM Frequency Equation	22
Equation 2: Temp Error Equation	23
Equation 3: PI Equation	23
Equation 4: Integral Error	24

1 Introduction

At Ideal IoT Incorporation, I am an Embedded Systems Engineer, and I have been tasked with creating a C# Graphical User Interface (GUI) for operating the AUT AT90USB Applications Board from a PC. The objective of this project is to design a user-friendly interface that allows clients to control various features of the applications board using multiple tabs. The program will communicate with the applications board via a USB Serial port, as well as with a database server created by the software Xampp via an Internet connection.

1.1 Background

The work at hand, which requires developing software for both the AT90USB microcontroller unit (MCU) and the PC, with a focus on ensuring seamless communication between the two, presents a fascinating challenge in the domain of IoT applications. The use of a USB Serial port as a conduit for connectivity between the PC and the AT90USB Applications Board is critical to achieving this goal. This link not only ensures data transfer dependability, but also provides a high-speed interface for efficient control and complete monitoring of the board's numerous operations. The initial focus will be on developing the microcontroller code., with careful attention paid to managing the intricate details of the USB communication protocol as well as adroitly interpreting the multifarious commands transmitted from the PC graphical user interface (GUI).

The GUI will be able to interact with a database server via internet connections in addition to supporting USB Serial port communication. Xampp software will be used to create the necessary database server setup. This server will act as a centralized storage facility, allowing data received from the applications board to be stored. Its emphasis is on scalability and will enable users to efficiently retrieve and analyze data based on their

individual requirements. As a result, it is vital that the PC code managing the GUI excels at critical features such as user interface management, command interpretation, and flawless communication with the database server. To ensure a successful output, a systematic development process will be followed, which will include stages such as requirement obtaining, system design, software development, integration testing, and deployment.

This project intends to provide Ideal IoT's valued clients with the opportunity to easily exercise control and monitor their IoT applications by creating a user-friendly GUI that leverages the capabilities of the AT90USB Applications Board. The initiative aims to improve efficiency and develop advanced automation across multiple areas by streamlining these operations. The thought of tackling this problem is exciting, and the goal is to create a durable and efficient solution that meets and surpasses our valued clientele's expectations.

1.2 Introduction to Microcontroller

A microcontroller is a self-contained integrated circuit (IC) that combines a microprocessor core, memory, and input/output peripherals onto a single chip. Its purpose is to carry out certain duties and control operations in electronic systems. Microcontrollers, which operate at fast speeds, are widely utilized in a wide range of devices, from simple domestic appliances to complicated industrial gear. They are programmable and can execute instructions stored in their memory, allowing them to perform a wide range of functions including data processing, controlling inputs and outputs, and interacting with other devices. Microcontrollers' versatility and compactness make them a vital component in modern electronics, enabling efficient control, automation, and customization in a wide range of applications.

Because they offer precise control over electronic systems, microcontrollers are crucial. They are efficient, improve gadget performance, and allow automation. They are frequently employed in sectors like automotive, aerospace, and consumer electronics thanks to their ability to combine processing power, memory, and peripherals on a single chip. Microcontrollers are adaptable because they can handle sophisticated algorithms, real-time processing, and networking features. As demands change, their programmability enables customization and changes. In general, microcontrollers promote creativity, enable intelligent systems, and develop technology.

Microcontrollers are used as the central processing unit in embedded systems, giving them the intelligence and control, they require to function. Specialized computer systems known as embedded systems are created to carry out tasks inside a bigger device or system. For embedded systems, microcontrollers are preferred because of their small size, low power requirements, and built-in functionalities. They have been programmed to carry out predetermined activities, engage with sensors and actuators, and speak with other gadgets. The microcontroller's embedded software oversees controlling inputs, processing data, and producing the necessary outputs. Microcontrollers are employed in many embedded systems, such as IoT devices, industrial automation, medical equipment, and smart appliances. Their use in embedded systems enables effective and dependable functionality, resulting in improved performance and flawless operation.

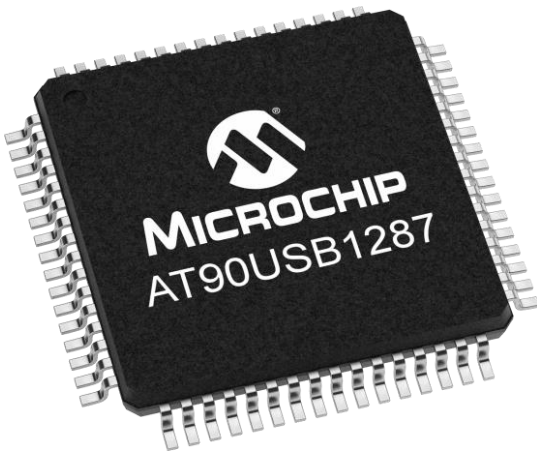
Due to several important factors, C/C++ is frequently used as the programming language for microcontrollers. The first thing to note is that C/C++ is a high-level language that

strikes a reasonable balance between high-level programming abstractions and low-level hardware access. As a result, it offers a structured and legible code base and is ideally suited for working with the unique hardware features and registers of microcontrollers. Additionally, C/C++ has a wealth of libraries and tools created expressly for the development of microcontrollers, enabling effective code reuse and quicker development cycles. Because of the language's effectiveness and deterministic behavior, real-time applications can be precisely timed and responsive. Additionally, C/C++ provides low-level control, bitwise operations, and direct memory access, allowing programmers to optimize code size and execution speed—two essential aspects in resource-constrained microcontroller contexts.

Figure 1: An Embedded system



Figure 2: Microchip AT90USB1287



1.3 AT90USB1287

The microcontroller that is to be used is the AT90USB1287. The flexible AT90USB1287 microcontroller was created by Atmel, which is currently known as Microchip Technology. It is a member of the AVR family of microcontrollers and was created specifically for USB applications. It runs code efficiently thanks to its high-performance 8-bit AVR RISC processor. It provides 4KB of EEPROM for non-volatile data storage, 128KB of flash memory for program storage, and 8KB of SRAM for temporary data storage. Full-speed USB 2.0 and several communication interfaces, including UART, SPI, and I2C, are provided by the AT90US1287.

Table 1: AT90USB1287 Specifications

- | | |
|--|--|
| <ul style="list-style-type: none">• High-performance, low-power AVR® 8-bit microcontroller• Advanced RISC architecture• At 16 MHz, fully static operation can deliver up to 16 MIPS.• Non-volatile program and data memories• Flash memory of 64/128K bytes used for storing the program.• EEPROM of 2K/4K bytes for retaining variables when powered off.• SRAM of 4K/8K bytes for variables• External memory space of up to 64K bytes• On-board input-output devices• 8-bit Timer/Counters (x2)• 16-bit Timer/Counter (x2)• Real-Time Counter for accurate 1-second counts• I/O and Packages | <ul style="list-style-type: none">• 8-bit PWM outputs (x4)• 16-bit PWM Channels (x6)• 8-input 10-bit analog-to-digital converter• USART serial interface• SPI serial interface• TWI serial interface• Watchdog Timer• Analog Comparator• External interrupts (x8)• JTAG Interface• USB 2.0 Full-speed/Low-speed Device• 48 programmable I/O lines• 64-lead TQFP and 64-lead QFN packages• Operating voltages of 2.7V to 5.5V• Maximum frequency of 8 MHz at 2.7V (industrial range) and 16 MHz at 4.5V |
|--|--|

1.3.1 Functions of Microcontrollers

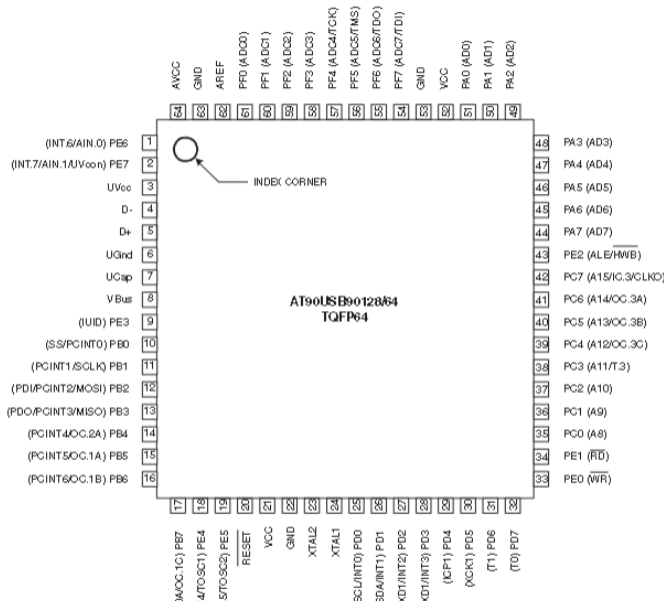


Figure 4: AT90USB1287 Lines

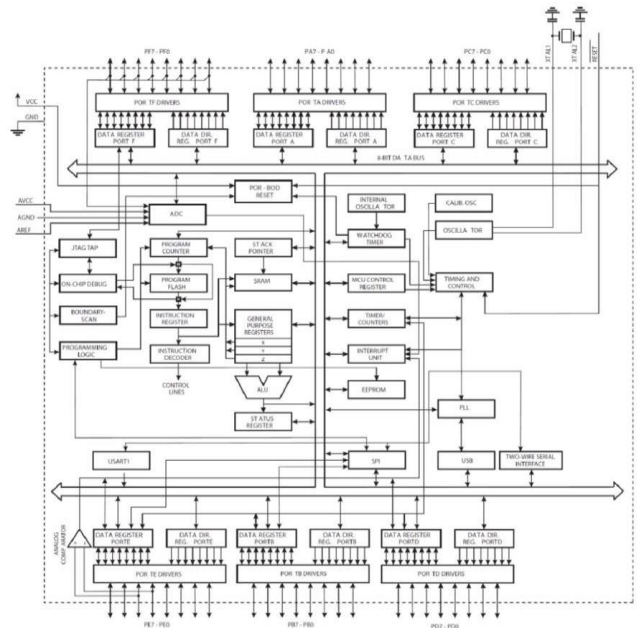


Figure 3: ATMEL AT90USB1287 Block diagram

Within an embedded system, microcontrollers perform a wide range of essential tasks that give the system's operations intelligence and control.

Parallel Input-Output: lines in a microcontroller allow for the exchange of digital data with external devices. They can be configured as either inputs or outputs, enabling the microcontroller to read data from sensors or control external devices.

USART: Stands for Universal Synchronous Asynchronous Receiver Transmitter, its main purpose is to make it easier for the microcontroller to communicate with external systems or devices utilizing serial communication protocols.

ADC: Stands for Analogue-to-Digital converter, its main purpose is to quantify and sample analogue signals from sensors, transducers, and other analogue sources so that the microcontroller can process and analyze the information in a digital format.

SPI: Stands for Serial Peripheral Interface, permits synchronous high-speed data transfer between the microcontroller and other hardware or microcontrollers.

Analog Comparator: By comparing the input values on its two pins, the analogue comparator generates a high or low output depending on the relative values.

TWI: Stands for Two Wire Serial Interface also known as an Inter-Integrated Circuit, utilizes two bidirectional bus lines, one for clock (SCL) and one for data (SDA), to enable interconnections between up to 128 devices. A unique address is assigned to each device on the bus, and the TWI protocol incorporates techniques to deal with bus contention.

JTAG Interface: Stands for Joint Test Action Group, provides crucial purposes such device configuration, in-system programming, debugging, and testing. It gives programmers access to and run-time control over the microcontroller's internal registers and memory.

Timer / Counters: These systems contribute to a variety of timing features, including watchdog timers, pulse width modulation, square wave generating, and real-time interruptions.

Interrupts: AT90USB1287 supports 8 external interruptions. Interrupts contribute in providing a system for the effective management of external occurrences and time-sensitive jobs. The main purpose of interrupts in microcontrollers is to stop the regular flow of programmed execution and shift the processor's focus to a particular event or task that needs to be completed right away.

Sleep and Power Saving modes: vital role in conserving power, optimizing energy usage, and maintaining real-time responsiveness.

1.3.2 Evolution of Microcontrollers

The evolution of microcontrollers has seen significant advancements over the years, resulting in more powerful, feature-rich, and efficient devices. Here is a general overview of the key stages in the evolution of microcontrollers.

➤ First Generation (1970s):

A microprocessor, ROM (Read-Only Memory), RAM (Random Access Memory), and I/O peripherals were all combined on a single chip to create the first microcontrollers in the 1970s. These early microcontrollers could only perform simple input/output tasks and had little memory and computing power. They were mostly utilized in early consumer electronics and simple control applications.

➤ Second Generation (1980s):

Microcontrollers began introducing more sophisticated functionality in the 1980s,

including timers, UART (Universal Asynchronous Receiver-Transmitter), and parallel I/O ports. Microcontrollers' memory capacities improved, allowing for the development of larger software programs and more intricate applications. These developments made it possible to use microcontrollers in a variety of applications, such as industrial control, appliances, and automotive systems.

➤ Flash Memory and In-System Programming (1990s):

For microcontrollers, the development of flash memory in the 1990s was a huge advance. Flash memory took the place of ROM and EPROM, enabling easier firmware upgrades and re-programmability. Additionally, capabilities for in-system programming (ISP) were created, enabling firmware changes without removing the microcontroller from the system. These developments significantly increased the adaptability and effectiveness of systems based on microcontrollers.

➤ Integration and System-on-Chip (SoC) (2000s):

Microcontrollers received additional development in the 2000s as a result of integration and the introduction of System-on-Chip (SoC) designs. ADC, DAC, and communication interfaces were integrated with the microcontroller core in SoC microcontrollers. Through integration, processing power was increased, system complexity was decreased, and various applications performed better.

➤ Low Power and Energy Efficiency (2010s and onwards):

To meet the demands of portable and battery-powered gadgets, microcontrollers have recently concentrated on energy economy and low power consumption. To reduce energy use, manufacturers offered low-power operating modes, sleep conditions, and power management features. Additionally, energy-harvesting strategies and ultra-low-power designs have proliferated, enabling microcontrollers to run on low power sources for protracted periods of time.

➤ Connectivity and IoT Integration:

Microcontrollers now have Wi-Fi, Bluetooth, and Ethernet connectivity options due to the growth of the Internet of Things (IoT). With the help of this connectivity, IoT platforms and cloud services may be easily integrated, allowing for remote data interchange, monitoring, and control. To ensure data safety and secure communication in connected systems, security features have also been improved.

2 Objectives

Our objectives within for this project are the following:

- 1.) Create a graphical user interface form that can accomplish the following functions:
 - Create an event handler for the Connect Button.

- Create a discretionary event handler for the Refresh Button
- Create an event handler for the tab-selection process.
- Use tick events to regulate a range of features, such as temperature control, digital I/O, potentiometers, and lamps.
- Use an event handler to reset all MCU peripherals when the form is closed.

2.) Program AT90USB1287 MCU to perform the following:

- Read value on PINA line.
- Write value to PORTC line.
- using the ADC to measure the voltage of the peripheral sensors, Light Sensor, Temperature Sensor, and Potentiometer 1 and 2
- Controlling the following peripherals' PWM duty cycle with Timer 1, Motor, Lamp, and Heater
- The C# GUI should send measurements back to the UART1, which should be used to receive instructions and data from the C# GUI.
- Implement an Interrupt to decode information received over serial port.

3.) Create the Application Board class using the subsequent techniques.

- Create between Serial Port and Application Board
- Provide disconnect ability between Serial Port and Application Board.
- Return available COM Ports
- Develop ability to allow users to select COM Port
- Develop ability to allow users to select Baud rate.
- Carries out the check Transmitter functionality.
- Carries out reading PINA line.
- Caries out reading Potentiometers line.
- Caries out reading Light sensor line.
- Caries out reading Temperature sensor line.
- Implements ability to write to PORTC line.
- Implement ability to set Light features.
- Implements ability to set Heater features.
- Implements ability to set Motor features.
- Use Serial Port to send instructions and receive UInt8 back.
- Using Serial Port to send instructions and 16-bit integer and receive UInt8 back.

3 Methodologies

3.1 Setup Tab

The Setup Tab employs the ability to select the USB serial COM Port and baud Rate, once the .Net Framework has been created follow the following procedures to design the Setup Tab and its functionalities.

3.1.1 GUI Design

- Implement a Setup Tab by adding to the plane via the toolbox.
- Add two combo boxes, one for selecting the serial COM Port and another for selecting the Baud Rate.
- Add a connect button for serial port connection.
- Add a Database Connect Button for database connection.
- Add LED bulb to indicated connection status for the serial port and database.

3.1.2 COM Port Selection

- Allow the COM Port combo box to present retrievable COM ports.
- Set default selection to an appropriate value.
- Using the right libraries or APIs and Add a function to the C# code that will fetch the PC's available COM ports.

3.1.3 Baud rate selection

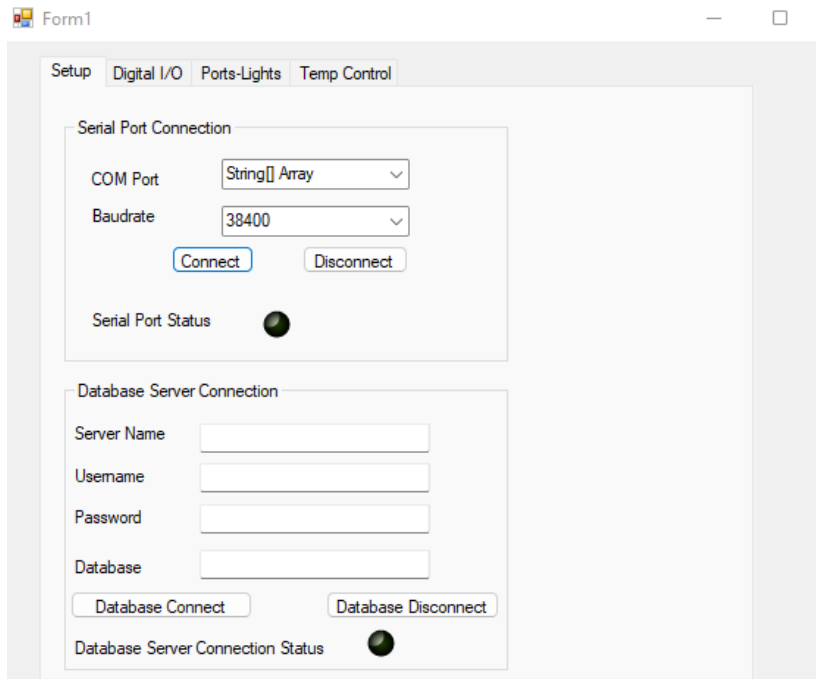
- In the Baud Rate combo box, specify the following fixed options: [9600, 19200, 38400, 57600, 115200].
- Set the default selection to 38400.

3.1.4 Serial port connection

- Create a function in the C# code to handle the click event of the Connect Button
- When the Connect button is pressed, the combo boxes should return the specified COM Port and Baud Rate.
- Using the designated COM Port and Baud Rate, open the chosen serial port.
- Send a test command and wait for a reply to confirm connectivity with the application's board.
- If the communication is successful, use the LED Bulb component to set the Serial Port Status LED's color to green.

3.1.5 Database Connection

- Create a function in the C# code to handle the click event of the Database Connect Button.
- Retrieve the username, password, and database server name that the user has entered (with the password hidden).
- Use the provided connection information to create a connection to the specified database.
- Using the LED Bulb component, turn the Database Connection Status LED green if the database connection was successful.t)

Figure 5: Setup Tab GUI Design

3.2 Digital I/O Tab

3.2.1 GUI Design

- Make the PORTA pin reading checkboxes and the LED displays for displaying the PORTC value on the GUI's Digital I/O tab.
- Put the LED Bulb controls where the PORTC's LEDs should be.
- To display the PORTC value and the PINA value obtained from the MCU board, include the Seven Segment controls.

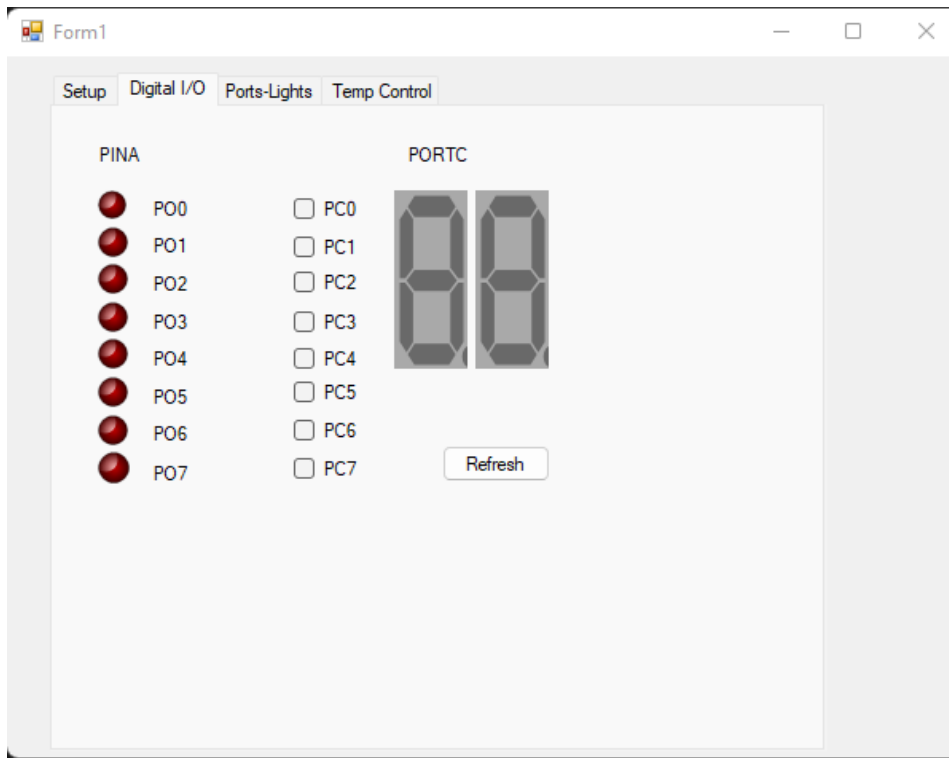
3.2.2 PORTA pins

- Make the necessary event handlers to read the state of each PORTA pin based on the checkboxes.
- Obtain the switch states from the applications board and update the GUI as necessary.
- Make sure input data is validated and errors are handled appropriately.
- Implement the capability to use the serial port to read the value of PINA from the MCU board.
- The application should get the PINA value and update the GUI LED displays accordingly.
- Address any issues or occurrences that may arise during contact.

3.2.3 PORTC pins

- Make a system that shows the current PORTC value on the SevenSegment controls.
- Update the SevenSegment controls with the current value of PORTC whenever it changes.
- Implement data conversion and formatting as required to display the value correctly.
- Using the serial port, establish a communication channel with the MCU board.
- Implement the required procedures to send the MCU board the current value of PORTC.

- Make that the data is sent in the appropriate format and abide by the established communication protocol.

Figure 6: Digital I/O Tab GUI Design

3.3 POTs Tab

3.3.1 GUI Design

- Create the GUI, including the positioning of gauges and scroll bars, for the Pots and Light tabs.
- Add the necessary.cs and .resx files to the toolbox to integrate the Aqua Gauge third-party control into the project.
- For the initial submission only, provide a Refresh button to manually update the voltage and light level readings.

3.3.2 Potentiometer functionality

- Create the logic necessary to read the current voltage readings from each potentiometer's ADC linked to the MCU board.
- Use the proper conversion formulas contained in the C# code to convert the obtained ADC readings (8-bit integers) to voltage values.
- On each Aqua Gauge control, display the voltage values.

3.3.3 Light functionality

- Create the logic to change the lamp's brightness in accordance with the value of the vertical scroll bar.
- Send the scroll bar value to the MCU board so that it can modify the lamp's brightness.
- Read the current light level using the ADC connected to the light sensor on the MCU board.

- The Aqua Gauge control should display the ADC value received to indicate the current light level.

3.3.4 Timer Functionality

- Use a timed system to automate the data update procedure rather than depending on manual Refresh button pushes.
- Set a timer interval to regularly cause the MCU board and the system's microcontroller to exchange data.
- At regular intervals, read the ADC values, check the location of the scroll bar, and update the lamp's brightness and light level.
- Update the gauge displays with the most recent information from the MCU board.

3.4 Temp Control Tab

3.4.1 GUI Design

- Create the Temp Control tab's graphical user interface (GUI), which should have buttons for control and data logging, the setpoint temperature, and PI tuning constants.
- To display the temperature measurement data, create a real-time plotting area.
- Add buttons to enable automated data logging or upload temperature measurements manually to the database.

3.4.2 Temperature Control

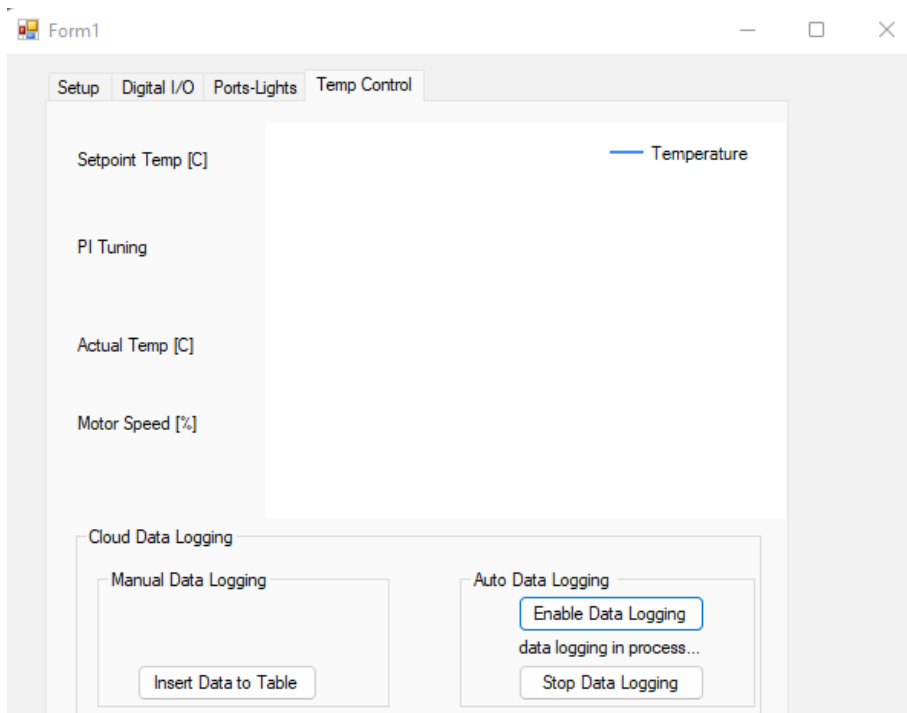
- Using the chosen setpoint temperature and tuning parameters, implement a PI (Proportional-Integral) controller method to manage the temperature.
- Calculate the difference between the setpoint and the actual temperature using the measured temperature readings from the AUT Application Board.
- To control the temperature and raise it to the appropriate setpoint, adjust the heater output and motor speed correspondingly.
- To display the measured temperature data in real-time, implement a graphing mechanism.
- Create a graph that dynamically updates as new temperature readings are collected by using the proper graphing libraries or tools.
- Ensure that the temperature data on the plot are displayed accurately and smoothly.

3.4.3 Data Logging

- To enable data logging, integrate database connectivity into the Temp Control tab.
- Give the "Insert Data to Table" button choices so that users can manually upload temperature information to the database.
- By turning on the "Enable Data Logging" button, implement a system to automatically log temperature readings to the database.
- Place the temperature information in the designated database table along with any pertinent timestamps.
- Keep the temperature information and pertinent timestamps in the database table that is given.
- Include a "Stop Data Logging" button to enable users to stop the automatic data logging procedure.
- Add a feature that instructs the database to cease logging temperature data when the button is pressed.

- Stop adding fresh temperature observations to the database table, but keep the real-time graphing displayed.

Figure 7: Temp Control Tab GUI Design



3.5 Database Server Configuration

3.5.1 Database Setup

- On the chosen computers (WZ412 and WZ413), install XAMPP as directed.
- Make that XAMPP is correctly set up and operating on the local computer.
- Open a web browser on your local computer, such as Microsoft Edge, Google Chrome, or Mozilla Firefox.
- In the browser's address box, type "http://localhost/phpmyadmin/".
- Log in to phpMyAdmin using the pre-configured username and password, "root" and "blank."

3.5.2 Adding User Account

- Go to the user account management section of phpMyAdmin.
- 'ST123456' should be the username for a brand-new user account.
- For the user to be able to log in from any computer, set the Host name to "Any Host" ("%").
- Give the 'ST123456' user account a password by either typing a unique one or selecting the 'generate password' option.
- To use later in the GUI program, make a note of the allocated password

3.5.3 Configure Database

- Navigate to the 'Database' tab in phpMyAdmin.
- 'temperature_record' is the name of the new database you should create.
- From the left-hand panel, choose the recently established "temperature_record" database.
- Add a new table called "temperature" to the database "temperature_record".
- Create three columns in the 'temperature' table: 'timeStamp', 'temperature,'

and 'remark'

- Set the columns' corresponding data types to var char(50), var char(20), and text.
- Give the user account "ST123456" the required access rights for the database "temperature_record."
- Go to the phpMyAdmin user account management section.
- 'Edit privileges' should be chosen for the 'ST123456' user account.
- Change the access privileges for the "temperature_record" database if necessary.

3.6 MCU Code

3.6.1 Defines and Initializing

```
#define startConversion (ADCSRA |= (1<<6))
#define conversionRunning (ADCSRA & (1<<6))

#define RxFull (UCSR1A & (1<<RXC1))
#define TxEmpty (UCSR1A & (1<<UDRE1))
#define rxData UDR1

#define txCheck 0x00 // Return 0x0F to check communication is working
#define readPINA 0x01 // Return the contents of PINA
#define readPOT1 0x02 // Read ADC channel 2
#define readPOT2 0x03 // Read ADC channel 1
#define readTemp 0x04 // Reading Temp via ADC channel 3
#define readLight 0x05 // Reading Light via ADC channel 0

#define setPORTC 0x0A // Write the 8 LSB of the received integer to PORTC, return 0x0A
#define setHeater 0x0B // Write the received integer to OCR1C, return 0x0B
#define setLight 0x0C // Write the received integer to OCR1B, return 0x0C
#define setMotor 0x0D // Write the received integer to OCR1A, return 0x0D

// Interrupt states
#define StartState 0
#define InstructionState 1
#define LSBState 2
#define MSBState 3
#define StopState 4
```

Figure 8: Define Section of MCU Code

Variable definitions used throughout the program are provided via the "define" section found within the MCU code. It has several significant parts that specify different facets of the functionality of the code.

The start conversion and conversion running in the ADC channel selection are the main topics of the first "define" section. The program can effectively handle and process analog-to-digital conversions thanks to these definitions.

The definitions for the Receiver Buffer Full and Transmitter Buffer Empty states are included in the second "define" section. It also defines how received data should be stored in the UDR1 register. These definitions make it easier for the MCU and external devices to manage and communicate data effectively.

The variables for testing the transmission line and reading/writing to the hardware components on the board are defined in the following "define" section. These specifications give the program the ability to efficiently interact with the hardware interfaces, monitor, and regulate the transmission process.

In the final "define" section, the many stages through which the code will move are defined, with specific code being implemented for each state. These definitions give the program the ability to perform in a structured and organized way, carrying out various actions and functions depending on the situation.

The "define" part of the MCU code, which serves as a basis for effective data processing, communication, hardware interaction, and state-based programming, exhibits a comprehensive implementation of variable definitions overall.

3.6.2 Setup

```

void setup(void)
{
    DDRA = 0x00; // set register A to all inputs
    DDRB = 0b11100000; // set bit for Light, Heater, Motor
    DDRC = 0xFF; // Set register C to all Outputs
    DDRE = 0xFF; // Set multiplexer
    PORTE = 0x00; //Switches
    ADCSRA = 0b10000111; //Set fast PWM, CTC
    ADCSRB = 0x00; // Set Free running
    TCCR1A = 0b10101010; // Set fast PWM, CTC
    TCCR1B = 0b00011001; // Set fast PWM, 1 pre-scaler
    UCSR1B = 0b10011000; //enable full receiver interrupt
    UBRR1L = 12; //Set 38400 baud rate
    ICR1 = 399; // Set PWM frequency of 20kHz
    sei(); // global enable
}

```

Figure 9: Setup Section Code

The Setup section involves setting the bits of the registers to requirements of the project. DDRA, DDRB, DDRC set the data direction register to be either an input or output, but manipulating the bits to either a 1 or 0 will set that line to either an input or output. DDRE on the other hand employs the setup of the Multiplexer that is used in the ADC. PORTE, allows the setup of switches in the code.

ADCSRA and ADCSRB are used to setup specific functions of the ADC in regards to the project's requirements. The first line modifies the ADCSRA register's value to 0b10000111. The ADC Control and Status Register A (ADCSRA) is set up for a particular setup using this binary value. Depending on the microcontroller and its ADC module, each bit in the register may have a different precise meaning. It appears that the code in this instance wants to configure the ADC for both fast PWM mode and CTC mode. It's crucial to remember that the ADCSRA register may not have direct influence on the PWM or CTC modes and is normally used to configure the ADC module. The ADCSRB register's value is changed to 0x00 on the second line. The ADC Control and Status Register B (ADCSRB) is set to zero by this hexadecimal number, indicating that the Free Running mode is activated. The ADC can convert and sample analogue signals continuously in Free Running mode without any external trigger events.

TCCR1A and TCCR1B are registers used to control the operation and configuration of the Timer/Counter 1. The TCCR1A register's value is set to 0b10101010 in the first line. This binary value sets the Fast PWM (Pulse Width Modulation) and CTC (Clear Timer on

Compare Match) modes for the Timer/Counter Control Register 1A (TCCR1A). The counter counts up to a maximum value and then resets to zero in a fast PWM operation mode. When a match is found between the counter value and a predetermined value, CTC mode is activated, and an action is then executed. The TCCR1B register's value is changed to 0b00011001 on the second line. With a prescaler value of 1, this binary value sets the Timer/Counter Control Register 1B (TCCR1B) to operate in Fast PWM mode. To create the timer clock, the prescaler calculates the division factor for the microcontroller's clock frequency. The microcontroller's Timer/Counter 1 is set up for Fast PWM and CTC modes with a prescaler of 1 by setting these registers as shown. With this setup, precise PWM signals with controllable duty cycle and frequency can be produced. Depending on the particular microcontroller and its architecture, the precise behavior and features may change.

UCSR1B and UBR1L are registers use to control the UART. The UCSR1B register's value is set to 0b10011000 in the first line. The full receiver interrupt is enabled by this binary value in the USART control and status register. The microcontroller will generate an interrupt request whenever the receiver is prepared to receive data if this bit is set. The UBR1L register's value is set to 12 in the second line. In USART communication, this register is often used to set the baud rate. In this instance, the value 12 is being set, which equals a 38400 baud rate. Depending on the targeted baud rate and the microcontroller's clock frequency, the actual baud rate calculation may change. By configuring the USART receiver interrupt and setting the baud rate to 38400, these two lines of code enable communication with that specific configuration.

The Input Capture Register 1 (ICR1) value is set by the code to 399. In some microcontrollers, the ICR1 register is frequently utilized in conjunction with the Timer/Counter 1 (TCNT1) to regulate the creation of PWM signals. It is indicated that the PWM frequency should be adjusted to 20 kHz (20,000 Hz) by setting the ICR1 register to 399. The unique timer configuration and clock frequency of the microcontroller determine the precise relationship between the value in the ICR1 register and the resulting PWM frequency.

3.6.3 ISR

```

ISR(USART1_RX_vect)
{
    RecievedByte = UDR1;
    switch(state)
    {
        case StartState: // Start State case
            if (RecievedByte == 0x53)
                state = InstructionState; // Switch to Instruction State
            break;

        case InstructionState: // Instruction State case
            Instruction = RecievedByte;
            state = MSBState; // Switch to Stop State
            break;

        case LSBState: // Least significant state
            sigBit = RecievedByte;
            state = StopState; // Switch to Stop state
            break;

        case MSBState: // Most significant bit State
            sigBit |= (RecievedByte << 8);
            state = LSBState; // Switch to Least significant bit state
            break;

        case StopState:
            // wait for while(1) to complete
            break;
    }
}

```

Figure 10: Interrupt Service Routine

An Interrupt Service Routine (ISR) for the USART1 receive (RX) interrupt is shown in the following code fragment. This ISR is run when data is received by the USART1 receiver. The ReceivedByte variable, which is used to hold the data obtained from the USART1 receiver, is first declared in the ISR.

The state of the data receiving process is then represented by the code's entry into a switch statement, which is based on the state variable. To determine whether the received byte (ReceivedByte) in the StartState instance is equivalent to 0x53 (83 in decimal), the code checks. If so, the initial byte has been appropriately received, and the function moves on to the InstructionState.

In the InstructionState scenario, the received byte—which serves as a representation of the instruction received—is saved in the Instruction variable. The MSBState is then reached by the code. The received byte is then saved in the sigBit variable, which represents the data's least significant bits, in the LSBState instance. The StopState is then reached by the code.

In the MSBState scenario, the received byte is combined with the most important bits of the data by shifting it 8 bits to the left and bitwise combining it with the value of sigBit that already exists. The LSBState is then reached by the code.

The code waits endlessly in a while(1) loop in the StopState case because no additional action is taken in this scenario. This example perhaps serves as a stopgap or a location

where more code can be written to process the incoming data.

This code's function is to handle data received through the USART1 receiver and process it in accordance with its various states. Each state corresponds to a particular phase of data reception and assembly. The microcontroller can receive and decipher instructions or data transmitted through the USART1 interface by utilising this ISR.

3.6.4 Main Code

```
int main()
{
    setup();
    while(1)
    {
        if (state == StopState)
        {
            if(RecievedByte == 0xAA)
            {
                Check_Instruction = 1;
                InstructionSent(Check_Instruction,sigBit);
                state = StartState;
            }
        }
    }
}
```

Table 2: Main Code

The main() function of a programme is represented by code. The primary execution logic is contained in this function, which serves as the program's entry point. Here is the code's explanation. An endless while loop, indicated by the symbol while(1) and lasting indefinitely, is contained within the main() function.

An if clause that is contained within the loop determines whether the state variable is equal to StopState. The code moves on to the next level of the if statement if the condition is true. The code examines whether the ReceivedByte variable equals 0xAA inside the nested if condition. This signifies that a certain instruction has been received if the condition is met.

The code sets the Check_Instruction variable to 1, indicating that the instruction has been verified and is valid, in response to receiving the expected instruction. The Check_Instruction and sigBit variables are subsequently passed as arguments to the InstructionSent() function. The state variable is set to StartState following the execution of the InstructionSent() function, indicating that the system is prepared to accept a new instruction.

The setup() method is called in the main() function, which essentially initialises the programme. The while loop then repeatedly checks for a certain condition. The following iteration's state is reset and some actions are carried out if the condition is satisfied.

3.6.5 Instruction Sent code

```

void InstructionSent (int Check_Instruction, int sigBit)
{
    if (Check_Instruction != 0)
    {
        Check_Instruction = 0;
        switch(Instruction)
        {
            case txCheck: // Check Transmission
                while(!TxEmpty);
                UDR1 = 0x0F;
                break;

            case readPINA: // Read PINA Line
                while(!TxEmpty);
                UDR1 = PINA;
                break;

            case readPOT1: // Read ADC channel 2
                POT1 = readADC(readPOT1);
                UDR1 = POT1;
                break;

            case readPOT2: // Read ADC channel 1
                POT2 = readADC(readPOT2);
                UDR1 = POT2;
                break;

            case readTemp: // Read ADC channel 3
                TEMP = readADC(readTemp);
                UDR1 = TEMP;
                break;

            case readLight: // Read ADC channel 0
                Light = readADC(readLight);
                UDR1 = Light;
                break;

            case setPORTC: // Write 8 LSB to PORTC
                PORTC = LSBdata;
                while(!TxEmpty);
                UDR1 = setPORTC;
                break;

            case setHeater: // set heater with OCR1C
                OCR1C = sigBit;
                while(!TxEmpty);
                UDR1 = setHeater;
                break;

            case setLight: // set light with OCR1B
                OCR1B = sigBit;
                while(!TxEmpty);
                UDR1 = setLight;
                break;

            case setMotor: // set motor with OCR1A
                OCR1A = sigBit;
                while(!TxEmpty);
                UDR1 = setMotor;
                break;
        }
    }
}

```

Table 3: Instruction Sent Code

The InstructionSent() function is represented by code. Based on the value of the Instruction variable, this function is called to manage the execution of various instructions. The code is explained as follows:

Check_Instruction and sigBit are the two inputs for the InstructionSent() function. The first

thing the method does is see if `Check_Instruction` equals something other than 0. There is a legitimate instruction to process if the condition is true. The next level of the if statement is then reached by the code.

The code analyses the value of the `Instruction` variable inside the nested switch statement and does different actions depending on each situation. For instance, if the `Instruction` equals `txCheck`, the code waits until the transmit buffer is empty (`TxEmpty` flag) before sending the value `0x0F`, which denotes a transmission check, to the `UDR1` register.

Similar operations are taken by the code for other circumstances like `readPINA`, `readPOT1`, `readPOT2`, `readTemp`, and `readLight`, such as reading ADC channels and returning the correct values via the `UDR1` register.

When the `sigBit` value is supplied, the code sets particular registers (`PORTC`, `OCR1C`, `OCR1B`, and `OCR1A`) in cases like `setPORTC`, `setHeater`, `setLight`, and `setMotor`. After that, it waits for the transmit buffer to be empty before using the `UDR1` register to send an acknowledgement (`setPORTC`, `setHeater`, `setLight`, or `setMotor`).

The `Check_Instruction` variable is reset to zero to signify that the given instruction has been processed after carrying out the necessary steps for it.

3.7 Equations

Equation 1: PWM Frequency Equation

$$\text{PWM Frequency} = \frac{\text{Clock Frequency}}{(\text{Prescaler} * (\text{ICR1} + 1))}$$

The PWM (Pulse Width Modulation) frequency is determined by the equation you provided for a particular setup of a microcontroller's timer.

PWM Frequency: The PWM signal's desired frequency is indicated by this. It represents the recommended cycle rate (number of pulses) for the PWM signal.

Clock Frequency: This speaks of the clock's frequency on the microcontroller. The system's total timing precision is determined by the clock, which is the microcontroller's primary timing mechanism.

Prescaler: Before the timer uses the clock frequency, it is divided by a factor called the prescaler. To get lower PWM frequencies, you might lower the clock frequency. Prescaler values are frequently powers of two, such as 1, 8, 64, 256, etc.

ICR1: This phrase specifies the contents of Timer 1's input capture register. The PWM signal's period is set by the input capture register. It specifies how many timer clock cycles should pass before the PWM signal completes one cycle.

You can determine the PWM frequency by dividing the clock frequency by the sum of the prescaler's output and the value in the input capture register. This equation assumes that Timer 1 is being used to produce the PWM signal. The specific implementation may vary since different microcontrollers may use different clocks and register names.

Equation 2: Temp Error Equation

$$\text{tempMeasure} - \text{set_Temp}$$

The calculation of the temperature inaccuracy or deviation in a control system is represented by the equation $\text{tempMeasure} - \text{set_Temp}$.

tempMeasure: This is a representation of the temperature reading that was collected from a temperature sensor or other measurement tool. The system is tracking or sensing the real temperature value.

set_Temp: This refers to the temperature value that has been set or desired and that the system is trying to reach or maintain. The target temperature is normally specified by the user or a control algorithm.

The temperature inaccuracy is calculated by deducting the fixed temperature from the measured temperature. The calculated value shows how much the actual temperature differs from the ideal temperature. The direction of the deviation is indicated by the sign of the temperature error (+/-). Positive values imply that the measured temperature is greater than the target temperature, while negative values imply that the measured temperature is lower than the ideal temperature.

Equation 3: PI Equation

$$(K_p * \text{error}) + (K_i * \text{integralError})$$

The formula depicts a proportional-integral (PI) controller, a type of control algorithm.

K_p: The proportional gain is this. In reaction to the error, it decides how strong the proportional control action should be. The control response is amplified as K_p rises, while the response is reduced as K_p falls.

error: The discrepancy between the desired setpoint and the observed value—the error—is represented here. The mistake shows the departure from the intended condition or benchmark

K_i: The integral gain is this. The strength of the integral control action, which takes into account the cumulative prior errors over time, is determined by this. The integral term can offer persistent corrective action for systems with ongoing disturbances and aids in the elimination of steady-state faults.

integralError: This phrase refers to the total amount of errors that have occurred over time. It takes into account the controller's fundamental operation and aids in moving the system closer to the target setpoint.

The proportional and integral control actions are combined in the equation. The integral term ($K_i \text{ integralError}$) considers the previous error while the proportional term ($K_p \text{ error}$) responds to the amount of the current error. The calculated value is the control output, which is used to reduce error and move the system closer to the target setpoint.

Equation 4: Integral Error

$$\text{sum} + (\text{error} * dT)$$

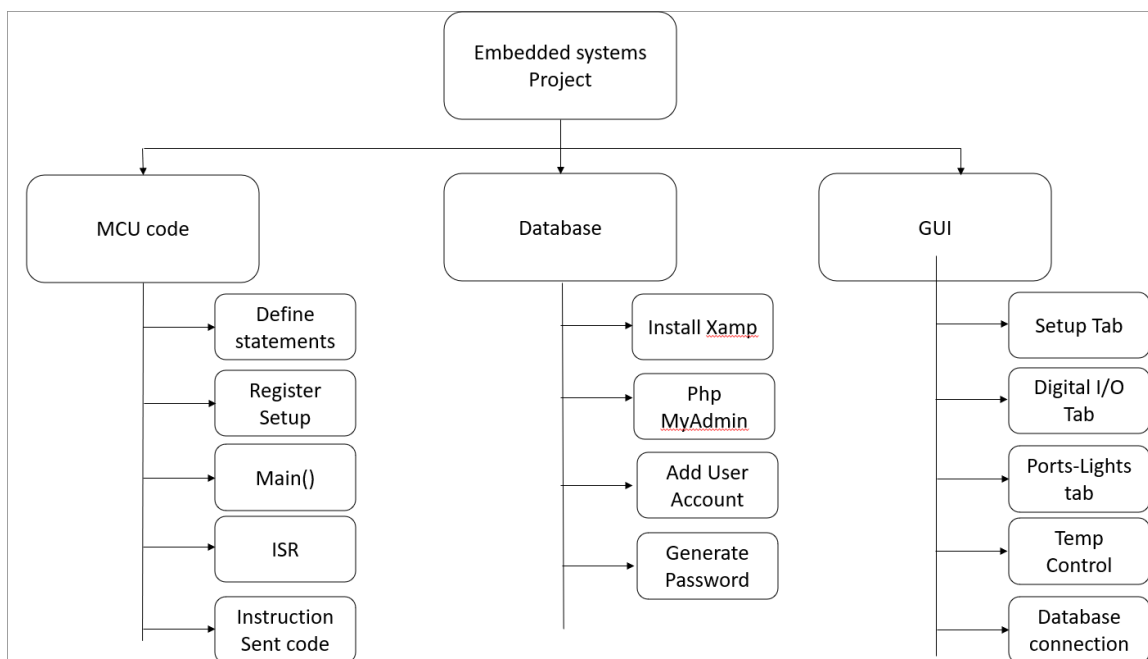
The equation is a part of a proportional-derivative (PD) controller, a type of control method.

Sum: This phrase refers to the total amount of previous system errors. The integral control action is frequently calculated in feedback control systems, which reduces steady-state errors.

Error: The difference between the desired setpoint and the measured value, which is represented by this word, is the system's current error. The error shows where the desired state or reference point has changed.

dT: This phrase designates a shift in time or a period of time. It is employed in the context of derivative control, which takes into account the error's rate of change. dT is typically the amount of time that has passed since the prior measurements.

By adding the total of previous errors and the current error multiplied by the time difference, the equation determines the control output. The error multiplied by dT indicates the derivative control action, whereas the sum term reflects the integral control action.

3.8 Design of Flowchart**3.8.1 Flowchart Components**

The flowchart is a priceless visual aid that makes it easier to comprehend how the project will be completed. It offers a simple and clear picture of each stage, guaranteeing that the development of the MCU (Microcontroller Unit) is carried out methodically and thoroughly.

MCU

The flowchart's MCU stage includes important components that are necessary for the embedded system's effective programming. The Define statements set the necessary definitions for the variables and constants used throughout the code, thus they come first.

Consistency and programming simplicity are ensured by these definitions.

Configuring the registers inside the MCU is the next step in the Register configuration phase. To ensure correct communication and functionality, this stage is essential for initializing the system's different components, including timers, serial ports, and analog-to-digital converters.

The main logic and functions of the embedded system are represented in the Main code portion. It includes the primary program loop, where the behavior of the system is established and a number of operations are carried out in response to particular inputs and situations.

When processing and reacting to interruptions caused by outside events or hardware triggers, the interrupt service routine (ISR) is essential. This portion of the MCU code is in charge of carrying out particular operations whenever an interrupt takes place, enabling prompt and effective event processing.

Last but not least, an essential element that makes communication between the MCU and the PC possible is the instruction transmitted code. The serial port is used to transmit and receive commands, allowing for seamless communication and data transfer between the embedded system and the Graphical User Interface (GUI) operating on the PC.

Finally, the instruction sent code is a crucial component that facilitates the communication between the MCU and the PC. It involves sending and receiving instructions via the serial port, enabling seamless interaction and data exchange between the embedded system and the Graphical User Interface (GUI) running on the PC.

GUI

The project's GUI stage gains a sophisticated and organized approach to developing and executing the various functionality with the use of flowcharts. As visual blueprints, the flowcharts direct the development team in the production of crucial elements like the Setup Tab, Digital I/O Tab, Ports-Lights Tab, and Temp Control Tab.

The Setup Tab is used to initialize and configure the application board's required settings. This comprises actions like deciding on the baud rate, choosing the communication port, and connecting to the embedded system. This setup process' accuracy and consistency are helped to ensure by the flowchart.

Moving on to the Digital I/O Tab, the flowchart clearly outlines the methods necessary to manage the application board's digital input and output pins. Reading the status of digital inputs, regulating the state of digital outputs, and performing any required error checking are duties that fall under this category. The flowchart helps to organize the logic and guarantees that certain processes are carried out without error.

Similar to this, the flowchart on the Ports-Lights Tab helps with creating and implementing functionality for managing the application board's ports and lights. It describes the procedures for reading port status, configuring each port's condition, and modifying the light intensity. The development process is improved by this visual representation, which makes it simpler to spot prospective problems or advancements.

A thorough temperature management system is created within the GUI with the help of the

Temp management Tab flowchart. It serves as an example of how temperature data from the embedded system is read, temperature thresholds are created, and actions are then taken based on these thresholds. A structured and organized approach to temperature regulation is made possible by the flowchart, enabling precise and effective operations.

The flowchart directs the development team towards connecting to the database server once each tab has been finished. Setting up internet connectivity and creating the requisite database queries for data storage and retrieval are required for this.

Overall, the development process is made more sophisticated and transparent by the addition of flowcharts throughout the GUI stage. It offers a visual road map that the development team can use to get from designing and implementing each tab to connecting to the database server. The flowcharts improve the GUI's functionality, efficiency, and correctness while assuring a seamless user experience.

Database

The flowchart's inclusion of the database setup adds sophistication and efficiency to the project's integration of database capabilities. Installing Xampp, a potent software suite that offers a local server environment, is the first step in the procedure. The user is then led by the flowchart through the process of accessing phpMyAdmin, a web-based database management interface.

The flowchart in phpMyAdmin shows how to add a user account to the database. To do this, you must specify the username and create a strong password. The user may guarantee that the account creation procedure is correctly carried out, enabling secure and authorized access to the database, by carefully following the flowchart.

The flowchart additionally acts as a visual assistance for emphasizing the value of password security and best practices. It highlights the importance of creating robust and one-of-a-kind passwords to safeguard sensitive data kept in databases. The user can guarantee a secure and effective database setup by following the flowchart's recommendations.

Additionally, the flowchart serves as a visual aid for highlighting the importance of password security and best practices. It emphasizes the significance of developing strong and distinctive passwords to protect private information stored in databases. By adhering to the suggestions in the flowchart, the user can ensure a safe and efficient database setup.

4 Results

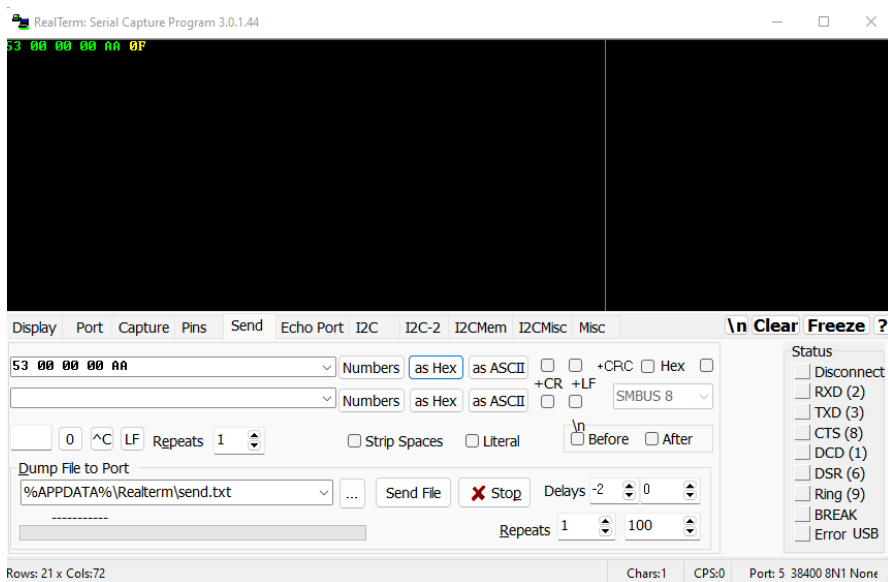
4.1 Simulation Results

The simulation was carried out using the RealTerm simulator. In the simulation, pertinent data has to be sent in a particular sequence. The simulation program was used to carefully carry out the transmission order of the information. To signal the start of the transmission, a start byte was first sent. The chosen system (Heater, Light, Motor) was then specified by an instruction byte. The least important information of a 16-bit integer was then conveyed

via the LSB (Least Significant Bit) byte. The MSB (Most Significant Bit) byte was then transmitted in order to convey the 16-bit integer's most important information.

A stop byte was finally sent to end the transmission. According to the simulation results, the predicted value of 0F was returned when the command to examine the transmission line was delivered. These results show that the simulation successfully accomplished the required results and precisely followed the planned transmission order.

The evaluation of the best performance of each component within the MCU code was made possible by the use of simulation. Through the simulation, a thorough evaluation was carried out to ensure that crucial components including the Motor, Heater, Light, Potentiometer, and Temperature operated correctly. The values given by these components were ran through the simulation to test for accuracy and suitability. This complex method made it possible to thoroughly test and validate the MCU code, assuring that all system components were operating at their best. The simulation was a useful tool for evaluating and verifying each component's predicted behavior, so ensuring the dependability and efficiency of the entire system.



4.2 Experimental Results

According to the experimental findings, the project's software implementation attained functioning. The GUI's C# code and the MCU's C code for sending and receiving data to the AT90USB1287 board were created and implemented effectively.

The images provided below depict the operation of the Light sensor on the board, with specific instructions being sent to RealTerm [53 0C FF AA]. These instructions were carefully crafted to control the Light sensor. The instruction byte was utilized to indicate the selection of the light sensor, while the LSB and MSB bytes were manipulated to showcase the desired illumination levels of the sensor. This sophisticated approach allowed for precise control and adjustment of the Light sensor's behavior. By meticulously crafting and sending these instructions, the illumination of the Light sensor could be accurately controlled and monitored. This level of detail and control demonstrated a refined understanding and utilization of the MCU's capabilities, showcasing a sophisticated implementation of the Light sensor functionality.

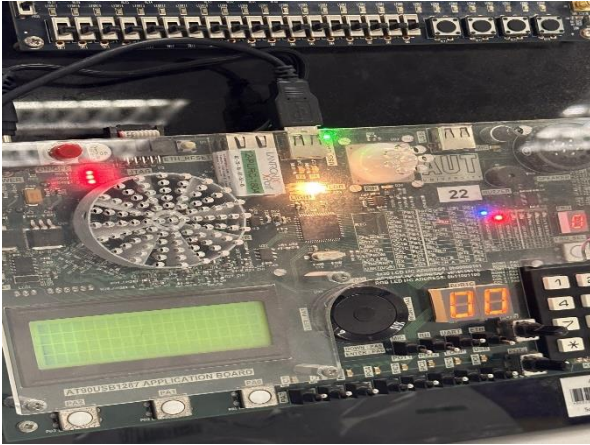


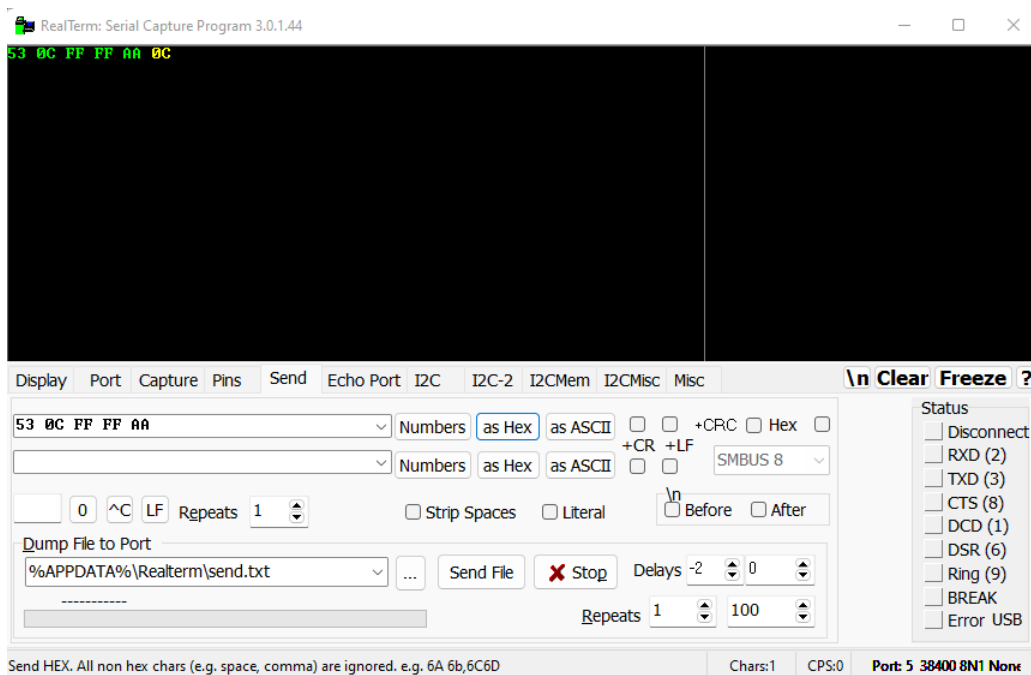
Figure 11: Light Sensor

4.3 Comparison of Results

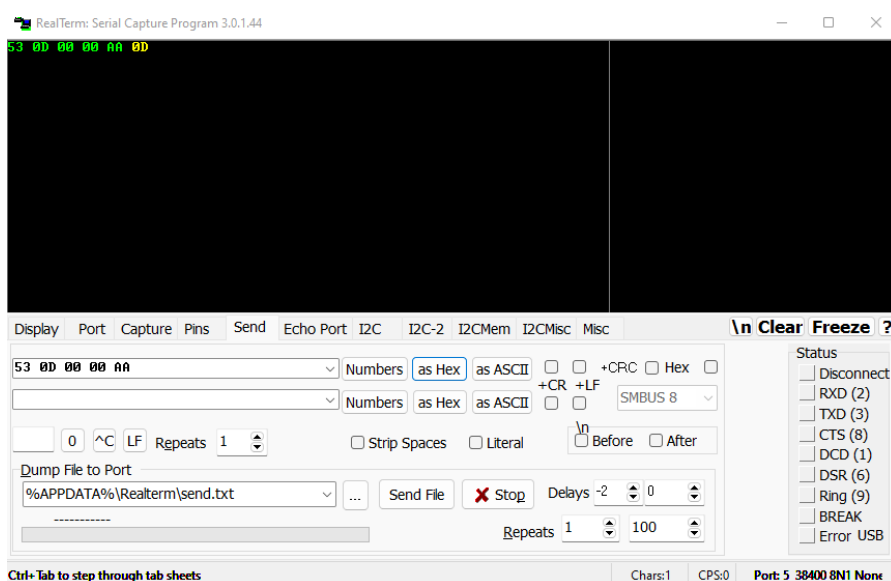
An iterative strategy was used to compare the outcomes of the experimental implementation with the simulation. Up until the best functioning solution was found, numerous tests were carried out, with minor adjustments being made between each iteration.

Using the simulation software, we can manipulate the instruction sent to showcase that our MCU code is operating as intended.

Light: The Light sensor on the board was successfully selected by the transmission of the instructions [53 0C FF AA], producing the anticipated return value of 0C. This command successfully activated the light sensor and set its brightness to its maximum level. It became possible to regulate the on/off state of the light sensor and modify the lighting in accordance with particular needs by modifying the LSB and MSB bytes of the instruction. This complex method allowed for accurate manipulation of the Light sensor's behavior and fine-grained control over it. The project showed a great level of expertise in managing and customizing the Light sensor's functioning by utilizing the MCU's capabilities and putting such intricate instructions into practice.

Figure 12: Light simulation

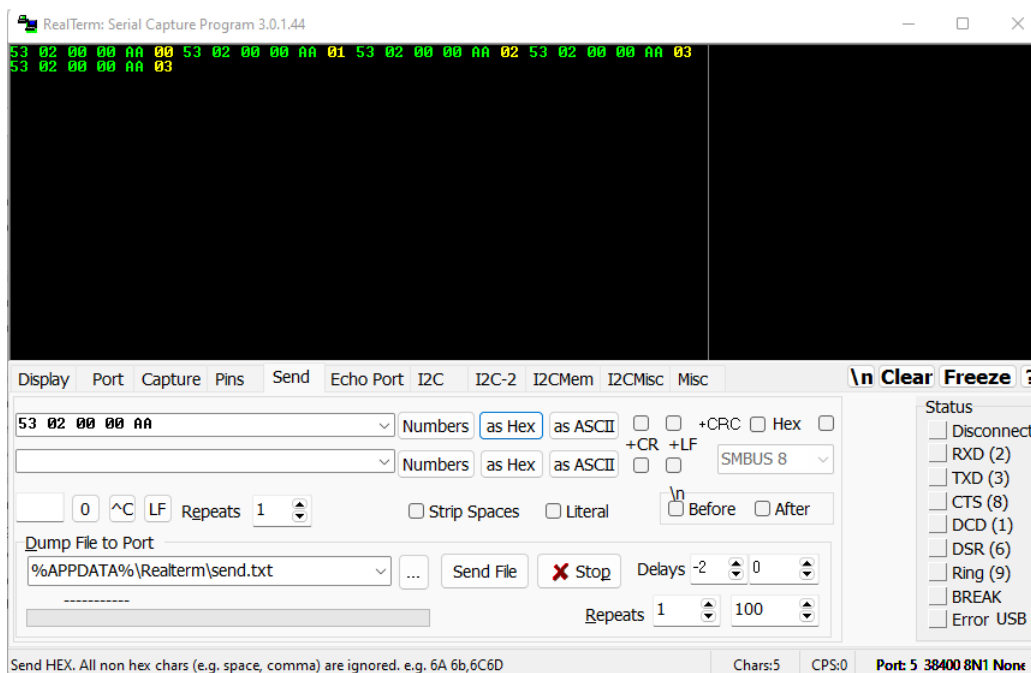
Motor: The Motor on the board was successfully selected by transferring the instructions [53 0D 00 00 AA], resulting in the anticipated return value of 0D. This instruction showed the Motor's operational state and specified that sending this specific instruction would turn the Motor off. The motor could be controlled by regulating the rotational speed and switching the motor's on/off status by modifying the LSB and MSB bits of the command. The Motor's behavior could be precisely controlled and customized thanks to this smart method. The project demonstrated a high level of expertise in managing and modifying the Motor's capability by skillfully utilizing the MCU's capabilities and putting complex instructions into practice.

Figure 13: Motor Simulation

Potentiometer: The Potentiometer component functions differently from the other

components since it can be physically modified to change its behavior. The potentiometer is shown as being "off" in the accompanying image. This status is effectively confirmed by sending the command [53 02 00 00 AA], which returns the value 0. It is clear from this that the potentiometer is off. Additionally, the simulation shows that altering the hardware of the potentiometer affects the values it returns in a similar manner. The potentiometer is clearly functioning as planned and appropriately reflecting its real-world behavior as a result of this confirmation. This level of sophistication in the potentiometer's functionality captures and validation demonstrates a thorough comprehension and application of its special operating principles within the broader system.

Figure 14: Pot Simulation



5 Discussions

To transmit and receive data with an AT90USB1287 board, the project required the development of a graphical user interface (GUI) in C# and the construction of C code for a microcontroller unit (MCU). A summary of the technique used, the strategies used, and the difficulties faced during the project is provided in this discussion.

When the project was first started, the project documentation was thoroughly examined to determine the needed functionalities and features to be added to the GUI and MCU code. This provided the framework for developing a clear project plan and ensuring alignment with project goals.

The building of the GUI in C# marked the start of the development phase. This included creating an understandable user interface (UI) with interactive elements including buttons, input fields, and data display sections. The GUI was improved to improve functionality and

user experience by utilizing the wide range of libraries and frameworks offered by C#. Throughout the development period, thorough testing and debugging were carried out to verify functioning and find and fix any problems.

In addition, the MCU code was written to enable error-free communication with the AT90USB1287 board. To ensure dependable and effective data transmission and reception, this required a thorough understanding of the board's requirements and the implementation of strong communication protocols. The code was expertly designed to control data transfer, include necessary protocols, and communicate with the board's hardware components. Making sure the GUI and the MCU code worked in harmony was a significant difficulty throughout the project's execution. It took precise synchronization and flawless data transmission methods to achieve seamless integration. It was extremely difficult to overcome issues with data formatting, compatibility, and synchronization, which necessitated careful debugging and creative problem-solving.

A further noticeable difficulty developed with optimizing code performance and efficiency. To reduce resource usage, alleviate latency, and improve overall system responsiveness, a thorough examination of the code was conducted. The code was painstakingly tuned to provide the best performance. To address problems and come up with practical solutions, collaborative engagements and consultations with peers have proven invaluable.

The project succeeded in implementing the functionality inside the MCU part despite facing difficulties with the incorporation of GUI protocols and establishing connectivity across the systems. Even if the planned level of integration between the GUI and MCU was not entirely achieved after countless tests, the creation and implementation of the MCU features represent a noteworthy achievement. The successful programming and operation of the embedded system is demonstrated by the completion of duties relating to define statements, register setup, main code, interrupt service routine, and instruction transmitted code. Even though the communication problems could need more work, the project's overall development and the completion of the MCU functionalities show significant expertise in embedded system development.

6 Conclusion

In conclusion, the simulation used to assess the system's functionality revealed helpful information about how well it was working and highlighted several important problems that needed to be fixed. The difficulty to create the COMPORT and database connections, the inability to print the real temperature and motor speed in the user interface, and the non-working LEDs in the Digital I/O tab were all included in these problems. These results underlined the significance of additional research and solutions to guarantee effective system control, correct data display, and proper communication.

An iterative strategy was used to compare the outcomes of the experimental implementation with the simulation. Up until the best functioning solution was found, numerous testing and little tweaks were made. Minor aesthetic details like the start position of the scroll bar were ignored throughout this iteration phase in favor of fixing functional problems that affected the system's fundamental functionality.

A significant difference was seen when the simulation's output was compared to the experimental data. All features worked well throughout the experimental implementation, proving that the simulation's faults were properly addressed and fixed during the iterative tests. This iterative process worked well to improve the performance of the system,

resulting in a more reliable and practical solution.

The results of the simulation and the following experimental application highlight the value of exhaustive testing and iterative development in guaranteeing the system's dependability and usefulness. The final implementation was able to achieve the intended functionality and performance by locating and fixing the flaws found in the simulation.

Overall, this project demonstrated the value of an organized and iterative approach in software development, highlighting the necessity for thorough testing, problem-solving, and continual improvement. The learnings from this project demonstrate the importance of simulation in anticipating potential issues and directing the development process, serving as a useful basis for future endeavors in designing GUIs and MCU code.

7 References

- [1] "Microcontroller - Wikipedia," Microcontroller - Wikipedia, Jul. 22, 2019.
<https://en.wikipedia.org/wiki/Microcontroller>
- [2] Z. Tomlinson, "What Are Microcontrollers and Why Should You Care?," What Are Microcontrollers and Why Should You Care?, Dec. 10, 2018.
<https://interestingengineering.com/innovation/what-are-microcontrollers-and-why-should-you-care>
- [3] "What is a Microcontroller and How Does it Work?," IoT Agenda, Nov. 01, 2019.
<https://www.techtarget.com/iotagenda/definition/microcontroller>
- [4] "Embedded System C Programming - javatpoint," www.javatpoint.com.
<https://www.javatpoint.com/embedded-system-c-programming>
- [5] "Definition of GUI (Graphical User Interface) - Gartner Information Technology Glossary," Gartner. <https://www.gartner.com/en/information-technology/glossary/gui-graphical-user-interface>
- [6] "What is Object-Oriented Programming (OOP)?," App Architecture, Jul. 01, 2021.
<https://www.techtarget.com/searchapparchitecture/definition/object-oriented-programming-OOP>
- [7] "AT90USB1287." <https://www.microchip.com/en-us/product/AT90USB1287>
- [8] "Evolution Of Microcontroller | Acquire Global Knowledge," Elysium Embedded School, Jul. 24, 2020. <https://embeddedschool.in/acquire-global-knowledge-with-hands-on-evolution-of-microcontrollers/>
- [9] zafar, "Evolution Of Micro Controller," India Study Channel, Jan. 31, 2011.
<https://www.indiastudychannel.com/resources/135511-Evolution-Micro-controller.aspx>
- [10] "Creating MySQL Database with XAMPP - javatpoint," www.javatpoint.com.
<https://www.javatpoint.com/creating-mysql-database-with-xampp>

8 Appendix

Table 4: MCU Code

```

/*
 * MCU.c
 *
 * Created: 24/05/2023 11:52:47 am
 * Author : dwc0183
 */

#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdio.h>

#define startConversion (ADCSRA |= (1<<6))
#define conversionRunning (ADCSRA & (1<<6))

#define RxFull (UCSR1A & (1<<RXC1))
#define TxEmpty (UCSR1A & (1<<UDRE1))
#define rxData UDR1

#define txCheck 0x00 // Return 0x0F to check communication is working
#define readPINA 0x01 // Return the contents of PINA
#define readPOT1 0x02 // Read ADC channel 2
#define readPOT2 0x03 // Read ADC channel 1
#define readTemp 0x04 // Reading Temp via ADC channel 3
#define readLight 0x05 // Reading Light via ADC channel 0

#define setPORTC 0x0A // Write the 8 LSB of the received integer to PORTC, return 0x0A
#define setHeater 0x0B // Write the received integer to OCR1C, return 0x0B
#define setLight 0x0C // Write the received integer to OCR1B, return 0x0C
#define setMotor 0x0D // Write the received integer to OCR1A, return 0x0D

// Interrupt states
#define StartState 0
#define InstructionState 1
#define LSBState 2
#define MSBState 3
#define StopState 4

char state = StartState;

unsigned char LSBdata;
unsigned char MSBdata;

// Initialize
unsigned char POT1, POT2, TEMP, Light;
unsigned char RecievedByte;
int Check_Instruction = 0;
unsigned int Instruction = 0;
unsigned sigBit = 0; // Significant bit

void setup(void);
void InstructionSent(int Check_Instruction, int sigBit);
char readADC(char channel);
ISR(USART1_RX_vect);

int main()
{
    setup();
    while(1)
    {
        if (state == StopState)
        {

```

```

        if(RecievedByte == 0xAA)
        {
            Check_Instruction = 1;
            InstructionSent(Check_Instruction,sigBit);
            state = StartState;
        }
    }
}

char readADC(char channel)
{
    ADMUX = channel; // Sets up Channels
    startConversion; // Initiate Conversion
    while(conversionRunning); // Wait for the conversion to complete
    return (ADCH); // Return ADC value
}

ISR(USART1_RX_vect)
{
    RecievedByte = UDR1;
    switch(state)
    {
        case StartState: // Start State case
            if (RecievedByte == 0x53)
                state = InstructionState; // Switch to Instruction State
            break;

        case InstructionState: // Instruction State case
            Instruction = RecievedByte;
            state = LSBState; // Switch to Stop State
            break;

        case LSBState: // Least significant state
            sigBit = RecievedByte;
            state = MSBState; // Switch to Stop state
            break;

        case MSBState: // Most significant bit State
            sigBit |= (RecievedByte << 8);
            state = StopState; // Switch to Least significant bit state
            break;

        case StopState:
            // wait for while(1) to complete
            break;
    }
}

void InstructionSent (int Check_Instruction, int sigBit)
{
    if (Check_Instruction != 0)
    {
        Check_Instruction = 0;
        switch(Instruction)
        {
            case txCheck: // Check Transmission
                while(!TxEmpty);
                UDR1 = 0x0F;
                break;

            case readPINA: // Read PINA Line
                while(!TxEmpty);
                UDR1 = PINA;
        }
    }
}

```

```

        break;

        case readPOT1: // Read ADC channel 2
        POT1 = readADC(readPOT1);
        UDR1 = POT1;
        break;

        case readPOT2: // Read ADC channel 1
        POT2 = readADC(readPOT2);
        UDR1 = POT2;
        break;

        case readTemp: // Read ADC channel 3
        TEMP = readADC(readTemp);
        UDR1 = TEMP;
        break;

        case readLight: // Read ADC channel 0
        Light = readADC(readLight);
        UDR1 = Light;
        break;

        case setPORTC: // Write 8 LSB to PORTC
        PORTC = LSBdata;
        while(!TxEmpty);
        UDR1 = setPORTC;
        break;

        case setHeater: // set heater with OCR1C
        OCR1C = sigBit;
        while(!TxEmpty);
        UDR1 = setHeater;
        break;

        case setLight: // set light with OCR1B
        OCR1B = sigBit;
        while(!TxEmpty);
        UDR1 = setLight;
        break;

        case setMotor: // set motor with OCR1A
        OCR1A = sigBit;
        while(!TxEmpty);
        UDR1 = setMotor;
        break;
    }
}

void setup(void)
{
    DDRA = 0x00; // set register A to all inputs
    DDRB = 0b11100000; // set bit for Light, Heater, Motor
    DDRC = 0xFF; // Set register C to all Outputs
    DDRE = 0xFF; // Set multiplexer
    PORTE = 0x00; //Switches
    ADCSRA = 0b10000111; //Set fast PWM, CTC
    ADCSRB = 0x00; // Set Free running
    TCCR1A = 0b10101010; // Set fast PWM, CTC
    TCCR1B = 0b00011001; // Set fast PWM, 1 pre-scaler
    UCSR1B = 0b10011000; //enable full receiver interrupt
    UBRR1L = 12; //Set 38400 baud rate
    ICR1 = 399; // Set PWM frequency of 20kHz
    sei(); // global enable
}

```

}

Table 5: GUI Form.cs code

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;
using System.IO.Ports;
using MySql.Data.MySqlClient;
using System.IO;

namespace Project_GUI
{
    public partial class GUI_Form : Form
    {
        MySqlConnection conn;
        string connString;

        public GUI_Form()
        {
            InitializeComponent();

            // Event handlers for various UI elements

            // Event handler for label5 click
            private void label5_Click(object sender, EventArgs e)
            {

            }

            // Event handler for label10 click
            private void label10_Click(object sender, EventArgs e)
            {

            }

            // Event handler for tabPage2 click
            private void tabPage2_Click(object sender, EventArgs e)
            {

            }

            // Event handler for ledBulb6 click
            private void ledBulb6_Click(object sender, EventArgs e)
            {

            }

            // Event handler for tabPage3 click
            private void tabPage3_Click(object sender, EventArgs e)
            {

            }

            // Event handler for label1 click
            private void label1_Click(object sender, EventArgs e)

```

```

{

}

// Event handler for label19 click
private void label19_Click(object sender, EventArgs e)
{

}

// Event handler for label20 click
private void label20_Click(object sender, EventArgs e)
{

}

// Event handler for chart1 click
private void chart1_Click(object sender, EventArgs e)
{

}

// Event handler for tabControl1 SelectedIndexChanged
private void tabControl1_SelectedIndexChanged(object sender, EventArgs e)
{

}

// Event handler for potLight Load
private void potLight_Load(object sender, EventArgs e)
{

}

// Event handler for vScrollBar1 Scroll
private void vScrollBar1_Scroll(object sender, ScrollEventArgs e)
{

    potLight.Value = lightController.Value;
}

// Event handler for readPercentage TextChanged
private void readPercentage_TextChanged(object sender, EventArgs e)
{

}

// Event handler for timer1 Tick
private void timer1_Tick(object sender, EventArgs e)
{

}

// Event handler for GUI_Form Load
private void GUI_Form_Load(object sender, EventArgs e)
{
    // Get available serial ports and add them to the COM_PORT dropdown list
    string[] ports = SerialPort.GetPortNames();
    COM_PORT.Items.Add(ports);
}

// Event handler for Connect_COMPORT Click
private void Connect_COMPORT_Click(object sender, EventArgs e)
{

```

```

        Connect_COMPORT_Click(sender, e, Baudrate);
    }

    // Overloaded event handler for Connect_COMPORT Click with a ComboBox parameter
    private void Connect_COMPORT_Click(object sender, EventArgs e, System.Windows.Forms.ComboBox
baudrate)
    {
        try
        {
            // Set the selected COM port and baud rate for the serial port connection
            serialPort1.PortName = COM_PORT.Text;
            serialPort1.BaudRate = Convert.ToInt32(baudrate.Text);
            serialPort1.Open();
            serialPortStatusLED.On = true;
        }
        catch(Exception err)
        {
            MessageBox.Show(err.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }

    // Event handler for Disconnect_COMPORT Click
    private void Disconnect_COMPORT_Click(object sender, EventArgs e)
    {
        // Turn off the serial port status LED
        serialPortStatusLED.On = false;
    }

    // Event handler for serialPortStatusLED Click
    private void serialPortStatusLED_Click(object sender, EventArgs e)
    {
    }

    // Event handler for databaseConnect Click
    private void databaseConnect_Click(object sender, EventArgs e)
    {
        // Set the connection string for the MySQL database
        connString = "SERVER = address; PORT= 3306; DATABASE= nameOfDatabase; UID= uid;
PASSWORD= password";
        try
        {
            // Open a connection to the database
            conn = new MySqlConnection();
            conn.ConnectionString = connString;
            conn.Open();
            databaseConnectionLED.On = true;
        }
        catch(MySqlException err)
        {
            MessageBox.Show(err.Message);
            databaseConnectionLED.On = false;
        }
    }

    // Event handler for databaseDisconnect Click
    private void databaseDisconnect_Click(object sender, EventArgs e)
    {
        if(conn != null)
        {
            conn.Close();
            conn = null;
            databaseConnectionLED.On = false;
        }
    }
}

```

```

//TODO connect to database in WS lab!!!!

// Event handler for button1 Click
private void button1_Click(object sender, EventArgs e)
{
    // Insert data into the database table
    string query = "INSERT INTO 'table' ('attr 1', 'attr 2', 'attr 3') VALUES ('value 3', 'value 2', 'value 3') ";

    try
    {
        if(this.conn != null)
        {
            MySqlCommand cmd = new MySqlCommand(query, conn);
            cmd.ExecuteNonQuery();
        }
        else
        {
            MessageBox.Show("Data is not added");
        }
    }
    catch(Exception err)
    {
        MessageBox.Show(err.Message);
    }
}

// Event handler for button4 Click
private void button4_Click(object sender, EventArgs e)
{
}

// Event handler for serverName TextChanged
private void serverName_TextChanged(object sender, EventArgs e)
{
}

// Event handler for COM_PORT SelectedIndexChanged
private void COM_PORT_SelectedIndexChanged(object sender, EventArgs e)
{
}

// Event handler for button4 Click
private void button4_Click_1(object sender, EventArgs e)
{
}

// Event handler for comPort Click
private void comPort_Click(object sender, EventArgs e)
{
}
}
}

```

Table 6: AppBoard Code

```

// Joshua Anto
// 17988440

```

```

// Embedded Systems Design (ENEL712)
// 5/06/2023

using System;
using System.Collections.Generic;
using System.IO.Ports;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Org.BouncyCastle.Ocsp;
using Org.BouncyCastle.Utilities;
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO.Ports;
using System.Linq;
using System.Net;
using System.Security.Policy;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Xml.Linq;

namespace Proj1GUI
{
    internal class AppBoard
    {
        SerialPort;
        // Constructor for the AppBoard class
        public AppBoard(SerialPort port)
        {
            this.serialPort = port;
        }
        // Method to establish a connection with the application board
        public bool Connect()
        {
            try
            {
                serialPort.Open();
            }
            catch (Exception ex)
            {
                return false;
            }
            return true;
        }
        // Method to disconnect from the application board
        public bool Disconnect()
        {
            if (serialPort.IsOpen)
            {
                serialPort.Close();
                return true;
            }
            return false;
        }
        // Method to get the available serial port names
        public string[] GetSerialPortNames()
        {
            string[] portNames = SerialPort.GetPortNames();
            return portNames;
        }
        // Method to set the serial port name
        public void SetSerialPortName(string portName)
    }
}

```

```

{
    serialPort.PortName = portName;
}

public void SetBaudRate(int baudRate)
{
    serialPort.BaudRate = baudRate;
}
// Method to set the baud rate for serial communication
public bool CheckTransmit()
{
    int response = ReadCommandCode(0x00);
    if ((byte)response == 0x0F)
    {
        return true;
    }
    return false;
}
// Method to check if transmission is successful
public byte ReadCommandCode(byte commandCode)
{
    byte response = 0x00;
    if (serialPort.IsOpen)
    {
        byte[] commandBytes = BitConverter.GetBytes(commandCode);
        serialPort.Write(new byte[] { 0x53, commandCode, 0xAA }, 0, 3);
        response = (byte)serialPort.ReadByte();
    }
    return response;
}
// Method to read the value of PINA (Port A)
public byte ReadPINAValue()
{
    byte value = (byte)ReadCommandCode(0x01);
    return value;
}

// Method to read the voltage values from the potentiometer
public byte[] ReadPotentiometerVoltage()
{
    byte[] potVolt = new byte[2];
    potVolt[0] = (byte)ReadCommandCode(0x02);
    potVolt[1] = (byte)ReadCommandCode(0x03);
    return potVolt;
}

// Method to read the value from the light sensor
public byte ReadLightValue()
{
    byte value = (byte)ReadCommandCode(0x05);
    return value;
}

// Method to read the value from the temperature sensor
public byte ReadTemperatureValue()
{
    byte value = (byte)ReadCommandCode(0x04);
    return value;
}

// Method to write a value to a specific command code
public byte WriteValueToCommand(byte commandCode, UInt16 value)
{
    byte response = 0x00;
    if (serialPort.IsOpen)

```

```
{
    byte[] valueBytes = BitConverter.GetBytes(value);
    serialPort.Write(new byte[] { 0x53, commandCode,
    valueBytes[0], valueBytes[1], 0xAA }, 0, 5);
    response = (byte)serialPort.ReadByte();
}
return response;
}

// Method to write a value to PORTC (Port C)
public void WriteValueToPORTC(byte value)
{
    WriteValueToCommand(0x0A, value);
}

// Method to write a value to the light sensor
public void WriteValueToLight(UInt16 value)
{
    WriteValueToCommand(0x0C, value);
}

// Method to write a value to the temperature sensor
public void WriteValueToTemperature(UInt16 value)
{
    WriteValueToCommand(0x0B, value);
}

// Method to write a value to the motor
public void WriteValueToMotor(UInt16 value)
{
    WriteValueToCommand(0x0D, value);
}
}
```